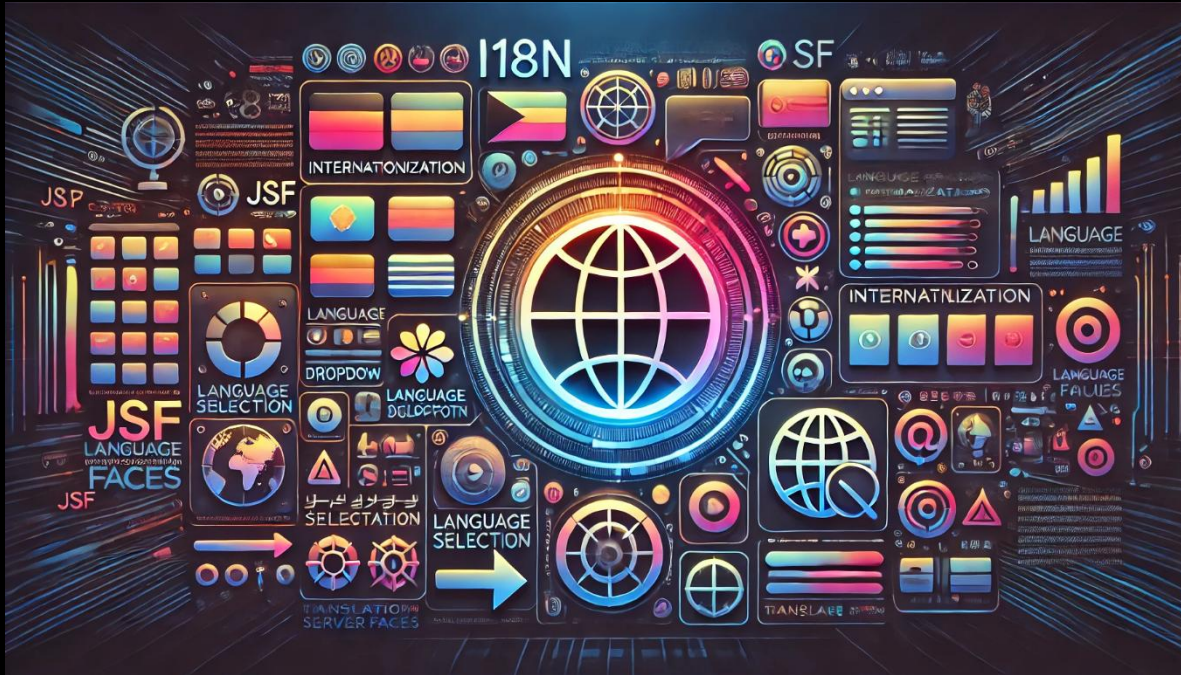


# Internacionalización de Mensajes en JSF



## Internacionalización de Mensajes en JSF

La **internacionalización (i18n)** en JavaServer Faces (JSF) permite que las aplicaciones sean compatibles con múltiples idiomas. Esto se logra mediante el uso de archivos de propiedades que contienen textos y mensajes localizados. En esta lección, aprenderás cómo configurar y utilizar la internacionalización en JSF utilizando la última versión del framework.

### 1. Concepto Básico

La internacionalización en JSF se basa en:

1. **Archivos de propiedades:** Contienen claves y valores para textos localizados.
2. **Configuración de `faces-config.xml`:** Define el idioma predeterminado y los bundles de recursos.
3. **Soporte dinámico:** JSF selecciona automáticamente el idioma basado en la configuración regional del navegador o del sistema.

Esto permite que la misma aplicación muestre contenido en diferentes idiomas dependiendo del usuario.

### 2. Configuración Básica de Internacionalización

## 2.1. Archivos de Propiedades

Los archivos de propiedades definen los textos localizados. Se crean múltiples versiones del archivo, una por cada idioma. Estos archivos deben colocarse en la carpeta `src/main/resources`.

### Archivo `mensajes_es.properties` (Español)

```
vacanteForm.nombre=Nombre
vacanteForm.apellido=Apellido
vacanteForm.salarioDeseado=Salario Deseado
vacanteForm.fechaNacimiento=Fecha de Nacimiento
vacanteForm.enviar=Enviar
```

### Archivo `mensajes_en.properties` (Inglés)

```
vacanteForm.nombre=Name
vacanteForm.apellido=Last Name
vacanteForm.salarioDeseado=Desired Salary
vacanteForm.fechaNacimiento=Birthdate
vacanteForm.enviar=Submit
```

---

## 2.2. Configuración de Mensajes de Validación

JSF permite personalizar mensajes de validación mediante un archivo adicional. Estos archivos deben colocarse en la carpeta `src/main/resources`. Por ejemplo:

### Archivo `jsf_es.properties` (Español)

```
jakarta.faces.component.UIInput.REQUIRED=Valor Requerido
```

### Archivo `jsf_en.properties` (Inglés)

```
javax.faces.component.UIInput.REQUIRED=Required Value
```

Estos archivos también deben colocarse en `src/main/resources`.

## 2.3. Configuración en `faces-config.xml`

El archivo `faces-config.xml` es donde configuramos:

- El idioma predeterminado.
- El bundle de recursos (`mensajes`).
- El bundle de validación (`jsf`).

### Código faces-config.xml

```
<?xml version='1.0' encoding='UTF-8'?>
<faces-config version="4.1"
    xmlns="https://jakarta.ee/xml/ns/jakartaee"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/web-facesconfig_4_1.xsd">
    <lifecycle>
        <phase-listener>
            beans.lifecycle.DebugListener
        </phase-listener>
    </lifecycle>
    <application>
        <locale-config>
            <default-locale>es</default-locale>
        </locale-config>
        <resource-bundle>
            <base-name>mensajes</base-name>
            <var>msgs</var>
        </resource-bundle>
        <message-bundle>jsf</message-bundle>
    </application>
</faces-config>
```

### Explicación:

1. **<default-locale>**: Define el idioma predeterminado (en este caso, es para español).
2. **<resource-bundle>**:
  - o **base-name**: Indica el nombre base del archivo de propiedades (sin extensión).
  - o **var**: Variable usada en los archivos `xhtml` y clases Java para acceder a los textos localizados.
3. **<message-bundle>**: Configura los mensajes de validación personalizados.

---

## 3. Ejemplo de Formulario Internacionalizado

### Archivo `index.xhtml`

El siguiente formulario utiliza internacionalización para mostrar textos en función del idioma configurado.

```

<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="jakarta.faces.html"
      xmlns:f="jakarta.faces.core">
  <h:head>
    <title>Bolsa de Trabajo V4</title>
  </h:head>
  <h:body>
    <h:form>
      <h:messages globalOnly="true" layout="table" styleClass="global-messages"/>

      <table>
        <tr>
          <td><h:outputLabel for="nombre"
                        value="#{msgs['vacanteForm.nombre']}" /></td>
          <td><h:inputText id="nombre" required="true"
                        value="#{candidato.nombre}" /></td>
          <td><h:message for="nombre" styleClass="error-message"/></td>
        </tr>
        <tr>
          <td><h:outputLabel for="apellido"
                        value="#{msgs['vacanteForm.apellido']}" /></td>
          <td><h:inputText id="apellido" required="true"
                        value="#{candidato.apellido}" /></td>
          <td><h:message for="apellido" styleClass="error-message"/></td>
        </tr>
        <tr>
          <td><h:outputLabel for="salarioDeseado"
                        value="#{msgs['vacanteForm.salarioDeseado']}" /></td>
          <td><h:inputText id="salarioDeseado" required="true"
                        value="#{candidato.salarioDeseado}"
                        <f:validateLongRange minimum="500" maximum="5000" />
          </h:inputText></td>
          <td><h:message for="salarioDeseado" styleClass="error-message"/></td>
        </tr>
        <tr>
          <td><h:outputLabel for="fechaNacimiento"
                        value="#{msgs['vacanteForm.fechaNacimiento']}" /></td>
          <td><h:inputText id="fechaNacimiento" required="true"
                        value="#{candidato.fechaNacimiento}"
                        <f:convertDateTime pattern="dd/MM/yyyy" />
          </h:inputText></td>
          <td><h:message for="fechaNacimiento"
                        styleClass="error-message"/></td>
        </tr>
      </table>
    </h:form>
  </h:body>
</html>

```

```

    </table>

    <h:commandButton action="#{vacanteForm.enviar}"
        value="#{msgs['vacanteForm.enviar']}" />
    </h:form>
</h:body>
</html>

```

## Explicación del Código:

- Uso de `msgs`:**
  - Los textos localizados se acceden con `#{msgs['clave']}`.
  - Ejemplo: `#{msgs['vacanteForm.nombre']}` muestra "Nombre" en español o "Name" en inglés.
- Mensajes de Validación:**
  - `<h:message>` muestra mensajes específicos para cada campo.
  - `<h:messages>` muestra mensajes globales.
- Cambio Dinámico de Idioma:**
  - Si el navegador del usuario está configurado en inglés, se usará `mensajes_en.properties`.
  - Si está en español, se usará `mensajes_es.properties`.
- Cambio Manual de Idioma:**
  - Modifica el archivo `faces-config.xml` según el idioma que desees probar, ej. `default-locale` con el valor de `en` para inglés o `es` para español:

## Modificar el archivo `Candidato.java`

Eliminamos cualquier mensaje que es muestre en el formulario dentro de la clase `Candidato.java`:

```

package beans.model;

import jakarta.enterprise.context.SessionScoped;
import jakarta.inject.Named;
import java.io.Serializable;
import java.util.Date;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Named("candidato")
@SessionScoped
public class Candidato implements Serializable {

    private final Logger log = LoggerFactory.getLogger(Candidato.class);

```

```
private String nombre;
private String apellido;
private int salarioDeseado;
private Date fechaNacimiento;

public Candidato() {
    log.info("Creando el objeto Candidato");
    //this.setNombre("Introduce tu nombre");
}

// Métodos getter y setter
public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
    log.info("Modificando la propiedad de nombre:" + this.nombre);
}

public String getApellido() {
    return apellido;
}

public void setApellido(String apellido) {
    this.apellido = apellido;
    log.info("Modificando la propiedad de apellido: " + this.apellido);
}

public int getSalarioDeseado() {
    return salarioDeseado;
}

public void setSalarioDeseado(int salarioDeseado) {
    this.salarioDeseado = salarioDeseado;
    log.info("Modificando la propiedad de sueldoDeseado: " + this.salarioDeseado);
}

public Date getFechaNacimiento() {
    return fechaNacimiento;
}

public void setFechaNacimiento(Date fechaNacimiento) {
    this.fechaNacimiento = fechaNacimiento;
}
```

}

## 4. Prueba y Verificación

1. **Cambia el idioma del navegador:**
  - Configura el idioma en inglés o español para verificar cómo cambian los textos automáticamente.
2. **Valida el formulario:**
  - Deja campos vacíos para ver los mensajes de validación localizados.

## 5. Conclusión

La internacionalización en JSF permite construir aplicaciones accesibles para audiencias globales. Utilizando archivos de propiedades, puedes administrar textos y mensajes localizados de manera eficiente, mientras que la configuración en `faces-config.xml` asegura que la selección de idioma sea automática y dinámica.

¡Con esta configuración, tu aplicación estará lista para ser usada por usuarios de diferentes regiones e idiomas!

Saludos!

**Ing. Ubaldo Acosta**

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)